Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2025-2026
Odd Semester

DATA STRUCTURES AND ALGORITHMS-3
(25CS2103E)

ROUTE OPTIMIZER USING BITMASK DP

K. YAMINI **2520030380**
L. LEKHANA **2520030364**

COURSE INSTRUCTOR

Swathi
Professor
Department of Computer Science and Engineering

ROUTE OPTIMIZER USING BITMASK DYNAMIC PROGRAMMING

1. Introduction

Finding the cheapest way to visit a set of places and return to the starting point is one of the most common optimisation problems in delivery services, logistics, field-service scheduling and tour planning. In graph theory this is the **Travelling Salesperson Problem (TSP)**: given a set of vertices and the cost of travelling between every pair of them, find the minimum-cost closed tour that visits every vertex exactly once. TSP is NP-hard, so no polynomial-time algorithm is known for the general case, but for a small number of locations an exact answer can be computed efficiently using **dynamic programming with bitmask state compression** (the Held–Karp approach).

This Problem Based Learning (PBL) project, **Route Optimizer**, applies that idea to a city-travel scenario. It is an interactive web application that models six areas of Hyderabad as a weighted graph, computes the optimal round trip with bitmask DP, and presents the outcome visually: the optimal route highlighted on the graph, the minimum cost in kilometres, and the DP states that were explored while solving it.

2. Objectives

- Represent a road network as a weighted, undirected graph using a distance (adjacency) matrix.
- Encode the set of visited locations as a single integer using a bitmask (state compression).
- Solve TSP exactly with dynamic programming, where dp[mask][current] stores the cheapest cost of reaching a location after visiting the locations in mask.
- Reconstruct the actual route from a parent table rather than only reporting its cost.
- Visualise the graph, the optimal tour and the DP states so that the working of the algorithm is easy to follow.

3. Case Study: Planning a Round Trip Across Hyderabad

A delivery executive must start from one area, visit every chosen area exactly once, and come back to the start while covering the least possible distance. The application offers six areas: Hyderabad, Secunderabad, Kukatpally, Madhapur, Gachibowli and LB Nagar. The user selects the start location and any subset of at least three areas. The distances (in km) between every pair of areas are stored in a symmetric matrix, so the graph is complete, with 6 vertices and 15 edges (n(n−1)/2).

Table 1: Distance matrix of the weighted graph (km)

	HYD	SEC	KUK	MDH	GCB	LBN
HYD	0	9	12	14	16	8
SEC	9	0	10	7	13	11
KUK	12	10	0	6	10	8
MDH	14	7	6	0	5	10
GCB	16	13	10	5	0	7
LBN	8	11	8	10	7	0

HYD = Hyderabad, SEC = Secunderabad, KUK = Kukatpally, MDH = Madhapur, GCB = Gachibowli, LBN = LB Nagar.

Example. With all six areas selected and Hyderabad as the start, the optimal tour is Hyderabad → LB Nagar → Gachibowli → Madhapur → Kukatpally → Secunderabad → Hyderabad, costing 8 + 7 + 5 + 6 + 10 + 9 = **45 km**. An arbitrary order such as Hyderabad → Secunderabad → Gachibowli → Kukatpally → Madhapur → LB Nagar → Hyderabad costs 9 + 13 + 10 + 6 + 10 + 8 = 56 km, and the most expensive of all 120 possible tours costs 71 km. Choosing the visiting order carefully therefore saves up to 26 km on a single trip in this data set.

4. Data Structures and Algorithmic Concepts Used

- **Weighted graph:** the six areas are vertices and the distances are edge weights, stored in a 6 × 6 two-dimensional array (distanceMatrix) with a zero diagonal.
- **Bitmask (state compression):** bit i of an integer is 1 when the i-th selected location has been visited. Bitwise shift, AND and OR operations test and update this set in constant time, and all 2^n subsets are covered by the integers 0 to $2^n - 1$.
- **Dynamic programming:** the problem has overlapping sub-problems and optimal substructure, so the minimum cost of every (visited set, current location) pair is computed once and reused.
- **Parent table and state log:** parent[mask][i] remembers the previous location of the best path so the route can be rebuilt by backtracking, and every improved DP state is logged for the on-screen DP table.
- **Index mapping:** a lookup object (globalToLocal) maps the chosen areas to indices 0 to n − 1 so that the mask needs only n bits, however many areas are selected.

5. System Overview

The application is written in plain HTML, CSS and JavaScript with no external libraries and runs by opening index.html in a browser. index.html defines the page layout and the SVG graph, style.css provides the styling and responsive behaviour, and script.js holds the graph data, the bitmask DP solver and the user-interface logic. The main features are:

- **Route Configuration panel:** a start-location drop-down, a checkbox for each of the six locations, and the Run Optimization and Reset Analysis buttons.

- **Weighted Route Network:** an SVG drawing of the six vertices and 15 labelled edges. After a run, the vertices and edges on the optimal tour are highlighted and the start vertex is marked separately.
- **Metric cards:** number of vertices, number of edges, size of the DP state space ($2^n \times n$) and the optimal cost.
- **Result cards:** the optimal tour as a sequence of locations, a state analysis (state space, states explored, final mask, complexity $O(n^2 \cdot 2^n)$) and the minimum cost in km.
- **Bitmask DP Execution table:** the first 12 logged DP states with their mask, visited locations, current location and cost.
- **Input validation:** an alert appears if fewer than three locations are selected or if the chosen start location is not among the selected locations.

6. Worked Example on a Small Instance

Take four locations, HYD, KUK, MDH and GCB, with Kukatpally as the start. Using local indices HYD = 0, KUK = 1, MDH = 2 and GCB = 3, the starting state is dp[0010][KUK] = 0. Table 2 shows every reachable DP value (a dash means the state cannot be reached).

Table 2: DP values dp[mask][current] for the four-location example (km)

Mask	Visited set	at HYD	at KUK	at MDH	at GCB
0010	{KUK}	–	0	–	–
0011	{HYD, KUK}	12	–	–	–
0110	{KUK, MDH}	–	–	6	–
0111	{HYD, KUK, MDH}	20	–	26	–
1010	{KUK, GCB}	–	–	–	10
1011	{HYD, KUK, GCB}	26	–	–	28
1110	{KUK, MDH, GCB}	–	–	15	11
1111	All four	27	–	33	31

To close the tour, the cost of returning to Kukatpally is added to each entry of the full mask 1111: HYD gives 27 + 12 = 39, MDH gives 33 + 6 = 39 and GCB gives 31 + 10 = 41. The minimum is **39 km**, and backtracking through the parent table gives Kukatpally → Madhapur → Gachibowli → Hyderabad → Kukatpally (6 + 5 + 16 + 12 = 39). Checking all 3! = 6 possible tours by brute force confirms that 39 km is the minimum.

7. Efficiency, Limitations and Applications

Brute force examines $(n - 1)!$ tours, whereas bitmask DP performs about $n^2 \cdot 2^n$ operations. For the six-location demo brute force is actually cheaper (120 tours), but the gap widens very quickly as locations are added:

Table 3: Brute force versus bitmask DP

Locations (n)	Brute force (n − 1)!	Bitmask DP $n^2 \cdot 2^n$
6	120	2,304
10	362,880	102,400
15	87,178,291,200	7,372,800
20	121,645,100,408,832,000	419,430,400

The method is exact but needs a table of $2^n \times n$ entries, so it is practical only for about twenty locations or fewer. The distances are fixed, symmetric sample values rather than live road data, and the drawn graph is fixed to six areas. The same technique applies to delivery routing, tour planning and field-visit scheduling.

ALGORITHM / PSEUDO CODE

1. Problem Formulation

Let G = (V, E) be a complete weighted graph on the n selected locations, let w(i, j) be the distance from the distance matrix, and let s be the start location. The aim is to find the minimum-cost cycle that starts at s, visits every other vertex exactly once and returns to s.

2. State Definition

A **mask** is an n-bit integer in which bit i is 1 if the i-th selected location has been visited. For n = 4, the mask 0111 means that locations 0, 1 and 2 are visited and location 3 is not.

- **dp[mask][i]** = minimum cost of a path that starts at s, visits exactly the locations in mask and ends at location i (i must be in mask).
- **parent[mask][i]** = the location visited just before i on that best path (−1 if none).

3. Recurrence Relation

- **Base case:** dp[1 << s][s] = 0, and every other entry is ∞.
- **Transition:** for every reachable state (mask, cur) and every next ∉ mask: dp[mask | (1 << next)][next] = min(dp[mask | (1 << next)][next], dp[mask][cur] + w(cur, next)).
- **Answer:** with full = $2^n - 1$, the minimum tour cost is min over i ≠ s of (dp[full][i] + w(i, s)).

4. Algorithm Steps

1. Read the selected locations and the start location from the interface. If fewer than three locations are selected, or the start is not selected, show an alert and stop.
2. Map the selected locations to local indices 0 to n − 1 (localToGlobal and globalToLocal) and let start be the local index of the start location.
3. Create the dp table (2^n rows, n columns, filled with ∞) and the parent table (filled with −1).
4. Set the initial mask to 1 << start, set dp[initialMask][start] = 0 and log this state.
5. For every mask from 0 to $2^n - 1$ and every current location in the mask whose dp value is finite, try each next location that is not yet in the mask.
6. Compute newMask = mask | (1 << next) and newCost = dp[mask][current] + w(current, next). If newCost is smaller than dp[newMask][next], store it, record parent[newMask][next] = current and log the state (relaxation).
7. After all masks are processed, look at the full mask. For every current location other than the start, add the cost of returning to the start and keep the smallest total together with its last location (finalNode).
8. Rebuild the route by starting from (fullMask, finalNode) and repeatedly following parent[mask][current] while removing current from the mask; place the start location at the front and append it at the end to close the cycle.

9. Return the cost, the route and the state log; the interface then fills the result cards, highlights the route on the SVG graph and builds the DP table.

5. Pseudo Code

```
FUNCTION SolveTSP(selectedNodes, startNode)
    n         <- length(selectedNodes)
    start     <- local index of startNode
    totalMask <- 2^n
    dp[0..totalMask-1][0..n-1]     <- INFINITY
    parent[0..totalMask-1][0..n-1] <- -1

    dp[1 << start][start] <- 0
    log state (1 << start, start, 0)

    FOR mask FROM 0 TO totalMask - 1 DO
        FOR cur FROM 0 TO n - 1 DO
            IF dp[mask][cur] = INFINITY THEN CONTINUE
            IF bit cur of mask is 0     THEN CONTINUE
            FOR next FROM 0 TO n - 1 DO
                IF bit next of mask is 1 THEN CONTINUE
                newMask <- mask OR (1 << next)
                newCost <- dp[mask][cur] + w(cur, next)
                IF newCost < dp[newMask][next] THEN
                    dp[newMask][next]     <- newCost
                    parent[newMask][next] <- cur
                    log state (newMask, next, newCost)
                END IF
            END FOR
        END FOR
    END FOR

    full <- totalMask - 1
    best <- INFINITY ;  last <- -1
    FOR cur FROM 0 TO n - 1 DO
        IF cur = start OR dp[full][cur] = INFINITY THEN CONTINUE
        total <- dp[full][cur] + w(cur, start)
        IF total < best THEN  best <- total ; last <- cur
    END FOR

    route <- empty list ; mask <- full ; cur <- last
    WHILE cur != -1 DO
        insert cur at the front of route
        prev <- parent[mask][cur]
        mask <- mask XOR (1 << cur)
        cur  <- prev
    END WHILE
    IF route[0] != start THEN insert start at the front of route
    append start to route
    RETURN (best, route, state log)
END FUNCTION
```

6. Controller Pseudo Code

```
FUNCTION OptimizeRoute()
    nodes <- indices of all checked locations ;  s <- chosen start
    IF length(nodes) < 3 THEN alert("Select at least 3 locations") ; RETURN
    IF s not in nodes    THEN alert("Start location must be selected") ; RETURN
    clear previous highlighting on the graph
    result <- SolveTSP(nodes, s)
    show cost, states explored, final mask and the route text
    highlight every vertex and edge of result.route on the SVG graph
    fill the DP table with the first 12 logged states
END FUNCTION
```

7. Correctness Argument

The recurrence is correct because of optimal substructure. If the cheapest path that visits the set mask and ends at i arrives from location j, then its first part must be the cheapest path that visits mask without i and ends at j; otherwise that part could be replaced by a cheaper one and the whole path would improve. Since masks are processed in increasing numerical order and every transition adds one bit (moving to a larger mask), all predecessor states are final before they are used. The result was also cross-checked against brute-force enumeration of all tours for the six-location and four-location examples in this report.

8. Complexity Analysis

- **Time:** there are 2^n masks, n choices of current location and n choices of next location, and each transition costs O(1) bit operations, so the running time is **$O(n^2 \cdot 2^n)$**.
- **Space:** the dp and parent tables each hold $2^n \times n$ entries, so the space is **$O(n \cdot 2^n)$**. The interface shows this number as "DP states"; it is $2^6 \times 6 = 384$ for six locations.
- **Route reconstruction:** backtracking visits at most n states, so it takes O(n) time.
- **States explored:** the interface counts each successful relaxation. This is 125 for the six-location run and 16 for the four-location example.

CODE

GitHub repository: https://github.com/yaminikatta18/Route-Optimizer-DSA

The repository contains three files: index.html (layout and SVG graph), style.css (styling) and script.js (data, solver and interface logic). The main logic from script.js is listed below, with the original blank lines and comment banners condensed for readability; the behaviour is unchanged. UI helper functions that only update the page (updateResultCards, updateDPTable, clearGraph, resetApplication) are not repeated here; they are in the repository.

script.js (core logic)

```
// ---------- 1. DATA : weighted graph as a distance matrix ----------
const locations = ["Hyderabad", "Secunderabad", "Kukatpally",
                   "Madhapur", "Gachibowli", "LB Nagar"];


const distanceMatrix = [
    /* HYD */ [0,  9, 12, 14, 16,  8],
    /* SEC */ [9,  0, 10,  7, 13, 11],
    /* KUK */ [12, 10, 0,  6, 10,  8],
    /* MDH */ [14, 7,  6,  0,  5, 10],
    /* GCB */ [16, 13, 10, 5,  0,  7],
    /* LBN */ [8,  11, 8, 10,  7,  0]
];

// ---------- 2. BITMASK HELPERS ----------
function isVisited(mask, v)  { return (mask & (1 << v)) !== 0; }
function addToMask(mask, v)  { return (mask | (1 << v)); }
function maskToBinary(mask, n) { return mask.toString(2).padStart(n, "0"); }


function getSelectedLocations() {
    const checked = document.querySelectorAll(".location-row input:checked");
    return Array.from(checked).map(c => Number(c.value));
}

// ---------- 3. TSP USING BITMASK DYNAMIC PROGRAMMING ----------
function solveTSP(selectedNodes, startNode) {
    const n = selectedNodes.length;
    const localToGlobal = selectedNodes;
    const globalToLocal = {};
    selectedNodes.forEach((g, i) => { globalToLocal[g] = i; });
    const start = globalToLocal[startNode];


    const totalMasks = 1 << n;
    // dp[mask][i] = minimum cost to stand at i having visited exactly 'mask'
    const dp     = Array.from({ length: totalMasks }, () => Array(n).fill(Infinity));
    const parent = Array.from({ length: totalMasks }, () => Array(n).fill(-1));
    const stateLog = [];


    // base state : only the start vertex visited
    const initialMask = 1 << start;
    dp[initialMask][start] = 0;
    stateLog.push({ mask: initialMask, current: start, cost: 0 });


    // DP transitions
    for (let mask = 0; mask < totalMasks; mask++) {
        for (let current = 0; current < n; current++) {
            if (dp[mask][current] === Infinity) continue;   // unreachable state
            if (!isVisited(mask, current)) continue;        // current must be in mask


            for (let next = 0; next < n; next++) {
```

```
                if (isVisited(mask, next)) continue;        // already visited

                const edgeCost = distanceMatrix[localToGlobal[current]][localToGlobal[next]];
                const newMask  = addToMask(mask, next);
                const newCost  = dp[mask][current] + edgeCost;

                // relaxation
                if (newCost < dp[newMask][next]) {
                    dp[newMask][next]     = newCost;
                    parent[newMask][next] = current;
                    stateLog.push({ mask: newMask, current: next, cost: newCost });
                }
            }
        }
    }

    // close the tour : best last vertex + edge back to the start
    const fullMask = totalMasks - 1;
    let minimumCost = Infinity;
    let finalNode = -1;

    for (let current = 0; current < n; current++) {
        if (current === start) continue;
        if (dp[fullMask][current] === Infinity) continue;
        const returnCost = distanceMatrix[localToGlobal[current]][localToGlobal[start]];
        const tourCost = dp[fullMask][current] + returnCost;
        if (tourCost < minimumCost) { minimumCost = tourCost; finalNode = current; }
    }

    // route reconstruction using the parent table
    const route = [];
    let mask = fullMask;
    let current = finalNode;
    while (current !== -1) {
        route.unshift(localToGlobal[current]);
        const previous = parent[mask][current];
        mask = mask ^ (1 << current);
        current = previous;
    }
    if (route[0] !== localToGlobal[start]) route.unshift(localToGlobal[start]);
    route.push(localToGlobal[start]);

    return { cost: minimumCost, route, stateLog, totalMasks, fullMask,
             statesPossible: totalMasks * n };
}

// ---------- 4. CONTROLLER : validation + UI update ----------
function optimizeRoute() {
    const selectedNodes = getSelectedLocations();
    const startNode = Number(document.getElementById("startNode").value);

    if (selectedNodes.length < 3) {
        alert("Select at least 3 locations for route optimization.");
        return;
    }
    if (!selectedNodes.includes(startNode)) {
        alert("The start location must be selected.");
        return;
    }
    clearGraph();
    const result = solveTSP(selectedNodes, startNode);
    latestResult = result;
    updateResultCards(result, selectedNodes);   // cost, states, mask, route text
    updateRouteGraph(result.route);             // highlight nodes and edges in SVG
    updateDPTable(result, selectedNodes);       // first 12 DP states in a table
}

// ---------- 5. GRAPH VISUALISATION ----------
function updateRouteGraph(route) {
    route.forEach((node, index) => {
```

```
        const el = document.getElementById("node-" + node);
        if (!el) return;
        el.classList.add("active");
        if (index === 0) el.classList.add("start");
    });
    for (let i = 0; i < route.length - 1; i++) highlightEdge(route[i], route[i + 1]);
}

function highlightEdge(a, b) {
    let edge = document.getElementById(`edge-${a}-${b}`) ||
               document.getElementById(`edge-${b}-${a}`);
    if (edge) edge.classList.add("active");
}
```

index.html (key elements)

```
<select id="startNode">
    <option value="0">Hyderabad</option>   <option value="1">Secunderabad</option>
    <option value="2">Kukatpally</option>   <option value="3">Madhapur</option>
    <option value="4">Gachibowli</option>   <option value="5">LB Nagar</option>
</select>

<label class="location-row">
    <input type="checkbox" value="0" checked>
    <span class="location-dot"></span><span>Hyderabad</span>
</label>                              <!-- one row per location (values 0 to 5) -->

<button class="optimize-btn" onclick="optimizeRoute()">Run Optimization</button>
<button class="reset-btn"    onclick="resetApplication()">Reset Analysis</button>

<svg id="routeGraph" viewBox="0 0 900 550">
    <g id="edges">
        <line id="edge-0-1" x1="130" y1="270" x2="300" y2="100"/>   <!-- 15 edges -->
    </g>
    <g class="graph-node" data-node="0" id="node-0">             <!-- 6 vertices -->
        <circle cx="130" cy="270" r="31"/>
        <text x="130" y="275">HYD</text>
    </g>
</svg>

<strong id="costMetric">--</strong>    <strong id="statesExplored">--</strong>
<div id="routeOutput" class="route-output"></div>
<tbody id="dpTable"></tbody>
```

style.css (key rules)

```
:root { --bg:#f5f7fa; --surface:#ffffff; --text:#111827; --primary:#315efb;
        --green:#16a36a; --sidebar:#111827; }
#edges line         { /* default grey weighted edge */ }
#edges line.active  { /* thick blue edge on the optimal route */ }
.graph-node.active circle { /* highlighted vertex on the route */ }
@media (max-width: 1150px) { /* responsive layouts */ }
@media (max-width: 850px)  { /* tablet */ }
@media (max-width: 600px)  { /* mobile */ }
```

RESULTS

Screenshots

The application was run in a browser and the following screens were captured. All values come from the program output.

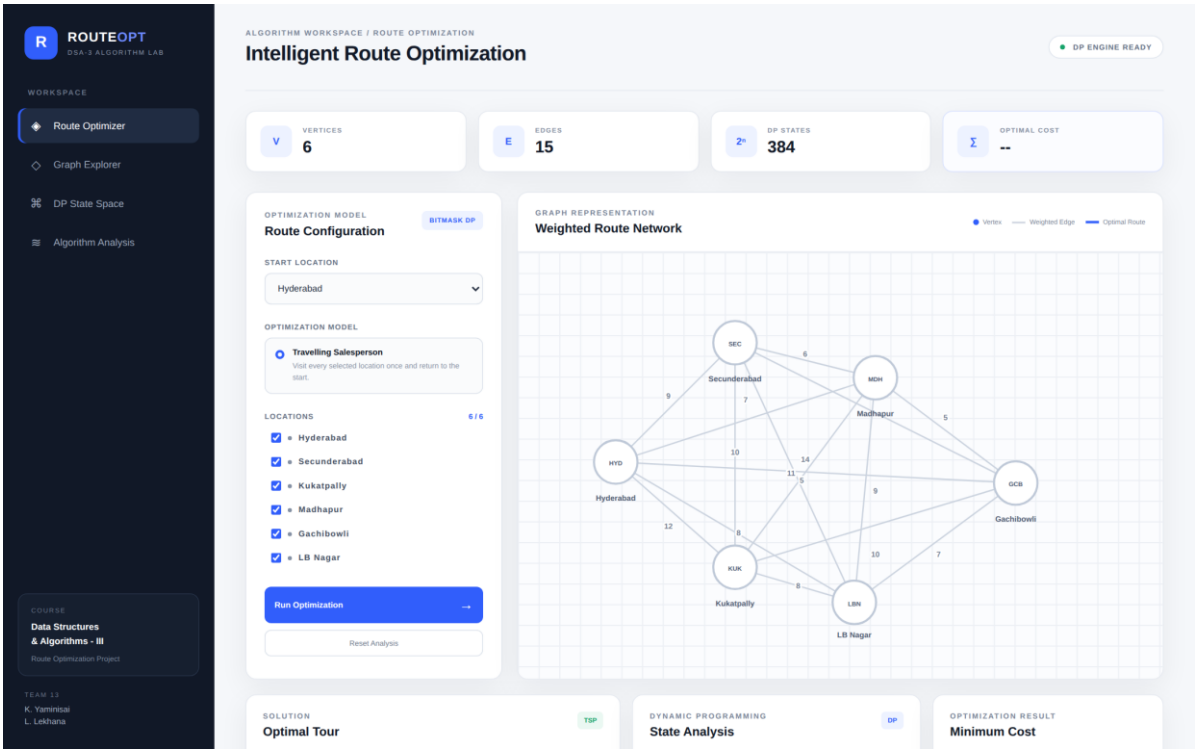


Figure 1: Initial screen with all six locations selected and Hyderabad as start

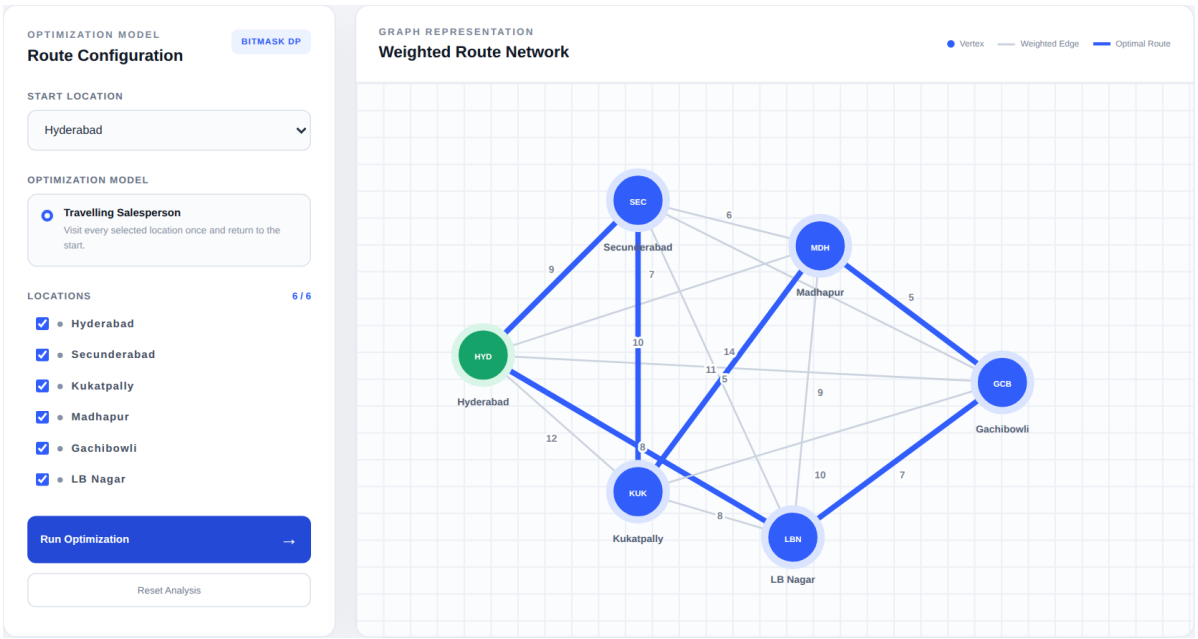


Figure 2: Optimal route highlighted on the weighted graph (6 locations, start Hyderabad)

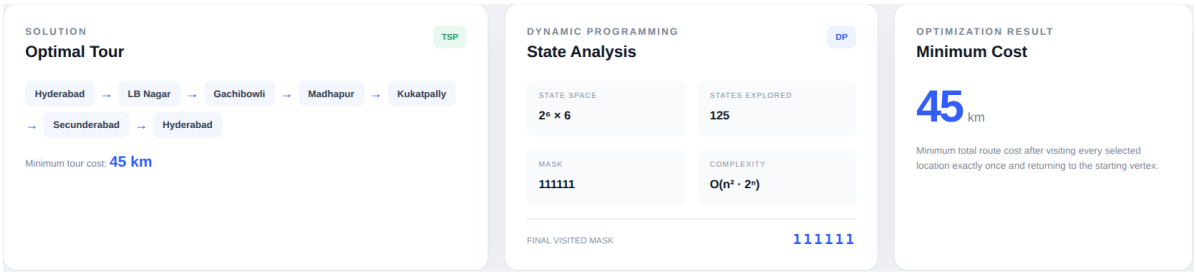


Figure 3: Optimal tour, state analysis and minimum cost (45 km)

STATE-SPACE REPRESENTATION

Bitmask DP Execution

`dp[mask][current]`

ITERATION	MASK	VISITED LOCATIONS	CURRENT	COST
1	000001	Hyderabad	Hyderabad	0 km
2	000011	Hyderabad, Secunderabad	Secunderabad	9 km
3	000101	Hyderabad, Kukatpally	Kukatpally	12 km
4	001001	Hyderabad, Madhapur	Madhapur	14 km
5	010001	Hyderabad, Gachibowli	Gachibowli	16 km
6	100001	Hyderabad, LB Nagar	LB Nagar	8 km
7	000111	Hyderabad, Secunderabad, Kukatpally	Kukatpally	19 km
8	001011	Hyderabad, Secunderabad, Madhapur	Madhapur	16 km
9	010011	Hyderabad, Secunderabad, Gachibowli	Gachibowli	22 km
10	100011	Hyderabad, Secunderabad, LB Nagar	LB Nagar	20 km
11	000111	Hyderabad, Secunderabad, Kukatpally	Secunderabad	22 km
12	001101	Hyderabad, Kukatpally, Madhapur	Madhapur	18 km

Figure 4: Bitmask DP execution table showing the first 12 logged states

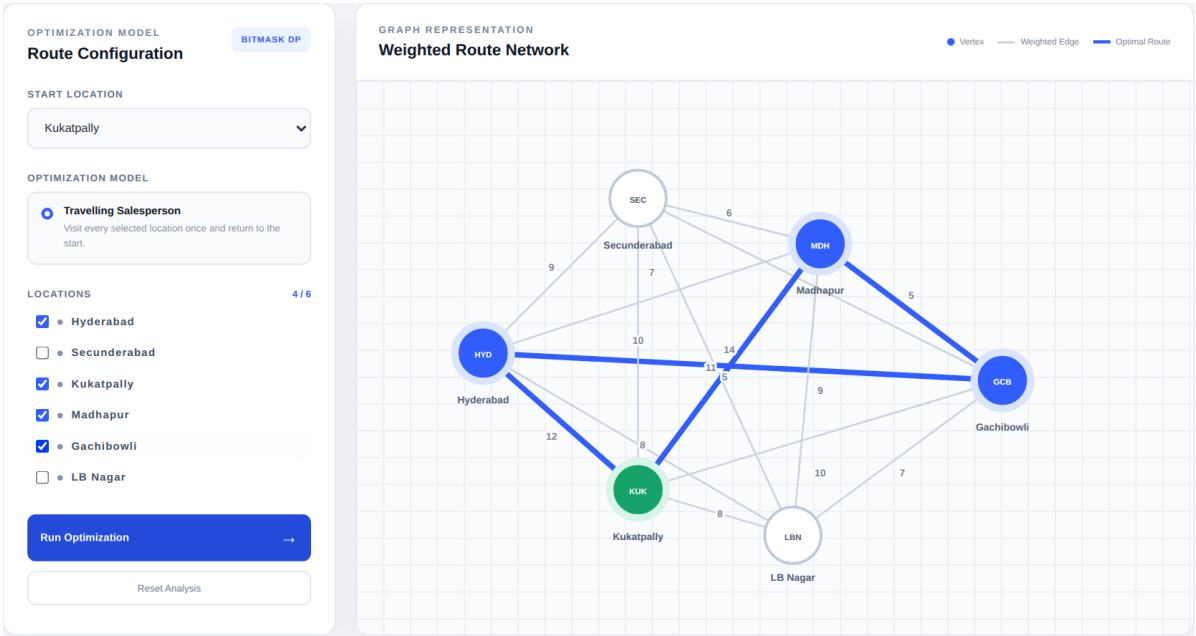


Figure 5: Four locations selected (Secunderabad and LB Nagar removed), start Kukatpally

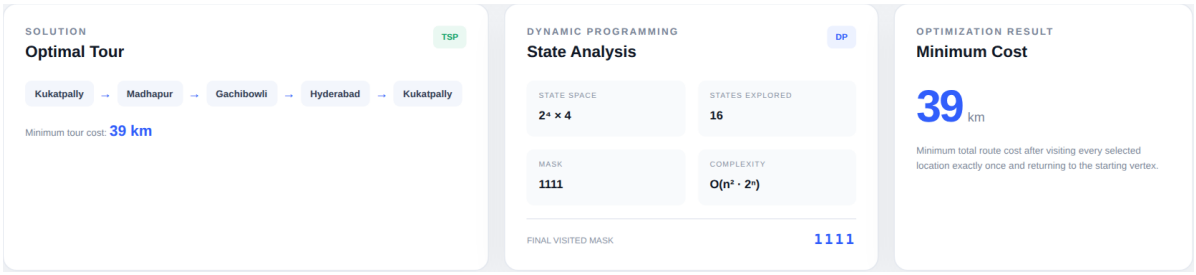


Figure 6: Result for the four-location case (39 km, mask 1111)

Summary of Results

Test case	Start	Optimal tour	Cost	States explored	Final mask
6 locations	Hyderabad	HYD → LBN → GCB → MDH → KUK → SEC → HYD	45 km	125	111111
4 locations (HYD, KUK, MDH, GCB)	Kukatpally	KUK → MDH → GCB → HYD → KUK	39 km	16	1111
Fewer than 3 locations	–	Alert: “Select at least 3 locations for route optimization.”	–	–	–

Observations and Conclusion

- Both optimal costs were verified by brute-force enumeration of every tour (120 tours for six locations, 6 tours for four locations), and the program output matched in each case.
- The optimal route found for the six-location case costs 45 km, compared with 56 km for an arbitrary order and 71 km for the worst possible order.
- The route reported by the program is the reverse direction of Hyderabad → Secunderabad → Kukatpally → Madhapur → Gachibowli → LB Nagar → Hyderabad, which has the same cost because the distances are symmetric.
- The project shows how bitmask state compression and dynamic programming turn an exponential search over all tours into an $O(n^2 \cdot 2^n)$ algorithm, and how a graph visualisation makes the DP states and the final route easy to understand.